CODESPECT

HyperHive

SECURITY ASSESSMENT REPORT

October 27th, 2026

Prepared for:

HyperHive

Contents

1 About CODESPECT **2**

2 Disclaimer **2**

3 Risk Classification **3**

4 Executive Summary **4**

5 Audit Summary **5**
5.1 Engagement Overview . . . 5
5.2 Scope – Audited Files . . . 5
5.3 Findings Overview . . . 6

6 System Overview **7**
6.1 Architecture: a fixed-rate vault backed by a strategy . . . 7
6.2 `HiveMarket` . . . 7
6.3 `IHiveStrategy` . . . 8
6.4 Trust model . . . 8
6.5 System Flow . . . 9

7 Findings **10**
7.1 Medium . . . 10
7.1.1 [Medium] Underfunded reward accrual locks the principal of all users . . . 10
7.1.2 [Medium] The amount returned by `strategy.withdraw(...)` is not verified . . . 11
7.2 Low . . . 12
7.2.1 [Low] `setApyBps(...)` with `needAccrue == false` applies the new rate retroactively . . . 12
7.2.2 [Low] `maxDeposit(...)`, `maxMint(...)`, `maxWithdraw(...)` and `maxRedeem(...)` ignore the pause and the solvency check . . . 12
7.3 Informational . . . 13
7.3.1 [Info] Frequent settlement compounds rewards above the configured APY . . . 13
7.3.2 [Info] One-step ownership transfer . . . 13
7.3.3 [Info] No upper bound on `apyBps` . . . 14
7.3.4 [Info] Code hygiene suggestions . . . 14

8 Threat Model **15**

9 Evaluation of Provided Documentation **18**
9.1 Documentation QA . . . 18

10 Test Suite Evaluation **19**
10.1 Compilation output . . . 19
10.2 Tests Output . . . 19
10.3 Cross-Cutting Weaknesses . . . 19
10.4 Specific Untested Paths . . . 19
10.5 Conclusion . . . 19

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensuring the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to building secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for **HyperHive**, an ERC-4626 Yield Vault on HyperEVM.

Audit Conclusion

CODESPECT assessed the in-scope contracts through manual review and proof-of-concept testing in Foundry. No Critical or High severity issue was identified. The HyperHive team fixed both Medium and both Low issues during the fix review, and CODESPECT verified each fix. Of the four Informational observations, three were fixed and one was acknowledged. The acknowledged one describes a property of the reward formula, not a flaw in the market logic. The market relies on trust assumptions that the team confirmed during the review, for example that the strategy does not lose principal and that the operator funds the rewards on time. Section 8 lists these assumptions. The report also gives recommendations for a test suite and for the documentation in Sections 9 and 10. The HyperHive team was very responsive throughout the review, answered all of our questions and provided correct fixes promptly. With the fix review complete and no finding left unresolved, CODESPECT considers the in-scope contracts ready for deployment, subject to the trust assumptions in Section 8 and the addition of a test suite.

The audit was performed using:

a) Manual analysis of the codebase.

b) Proof-of-concept tests in Foundry for the main findings.

c) Review of the code comments and of the build and test setup.

CODESPECT identified **eight points of attention**: **two** classified as `Medium`, **two** as `Low`, and **four** as `Informational`. All issues are summarised in the table below and detailed in Section 7.

Organisation of the document is as follows:

- **Section 3** presents the risk classification methodology and reference frameworks.
- **Section 5** summarises the audit: scope, schedule, coverage, and auditors.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the findings.
- **Section 8** presents the threat model.
- **Section 9** discusses the documentation provided by the client.
- **Section 10** discusses the test suite provided by the client.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Medium	0	2	0
Low	0	2	0
Informational	0	3	1
Total	**0**	**7**	**1**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

5.1 Engagement Overview

Audit Type	Security Review
Project Name	HyperHive
Type of Project	ERC-4626 Yield Vault
Duration of Engagement	1.5 days
Duration of Fix Review Phase	1 day
Initial Report	October 23rd, 2026
Final Report	October 27th, 2026
Repository	hyperhivefinance/Contracts
Commit (Audit)	`02b02ce7f8ad4b7d72e672322fb17a84751a190f`
Commit (Final)	`d9ba8e9311aae1ef971d60c5e05df9ca53f0aa68`
Documentation Assessment	Medium
Test Suite Assessment	Low
Auditors	jecikpo

Table 3: Summary of the Audit

5.2 Scope – Audited Files

	Contract	LoC
1	market/HiveMarket.sol	252
2	interfaces/IHiveStrategy.sol	12
	Total	**264**

5.3 Findings Overview

	Finding	Severity	Update
1	[Medium] Underfunded reward accrual locks the principal of all users	Medium	Fixed
2	[Medium] The amount returned by `strategy.withdraw(...)` is not verified	Medium	Fixed
3	[Low] `setApyBps(...)` with `needAccrue == false` applies the new rate retroactively	Low	Fixed
4	[Low] `maxDeposit(...)`, `maxMint(...)`, `maxWithdraw(...)` and `maxRedeem(...)` ignore the pause and the solvency check	Low	Fixed
5	[Info] Frequent settlement compounds rewards above the configured APY	Info	Acknowledged
6	[Info] One-step ownership transfer	Info	Fixed
7	[Info] No upper bound on `apyBps`	Info	Fixed
8	[Info] Code hygiene suggestions	Info	Fixed

6 System Overview

HyperHive is a single-asset yield market on HyperEVM. Users deposit an ERC-20 asset into an ERC-4626 vault and receive shares that accrue a reward at an annual rate, `apyBps`, set by the owner.

The audit covers two files: `market/HiveMarket.sol`, the vault, and `interfaces/IHiveStrategy.sol`, the interface through which the vault deploys its assets. The strategy implementation is out of scope and is treated as a trusted component. The OpenZeppelin v5 base contracts (`ERC4626`, `Ownable`, `Pausable`, `ReentrancyGuard`) are trusted primitives.

6.1 Architecture: a fixed-rate vault backed by a strategy

The system has three parties: the users who hold shares, the owner who operates the market, and the strategy that holds the deposited assets. `HiveMarket` keeps the accounting. The strategy runs on HyperEVM and returns assets in the same transaction when the market withdraws.

The share price does not follow the token balance of the market. It follows an internal counter, `accountedAssets`, which holds the deposited principal plus the rewards already recognized. On top of it, `totalAssets()` adds the reward that accrued linearly since the last settlement:

```
pendingReward = accountedAssets * apyBps * elapsed / (BPS * YEAR)
totalAssets() = accountedAssets + pendingReward
```

The strategy does not produce the reward on-chain. The operator harvests the strategy yield off-chain and sends tokens into the market as reward backing. A settlement recognizes the pending reward only if the market and the strategy together hold enough tokens to back it.

6.2 `HiveMarket`

`HiveMarket` is the ERC-4626 vault and the only stateful contract in scope. It overrides the four ERC-4626 entry points so that each one settles the pending reward before it changes balances (a zero amount returns 0 without settlement):

```
function deposit(uint256 assets, address receiver)
    public override nonReentrant whenNotPaused returns (uint256 shares) {...}
function mint(uint256 shares, address receiver)
    public override nonReentrant whenNotPaused returns (uint256 assets) {...}
function withdraw(uint256 assets, address receiver, address owner)
    public override nonReentrant whenNotPaused returns (uint256 shares) {...}
function redeem(uint256 shares, address receiver, address owner)
    public override nonReentrant whenNotPaused returns (uint256 assets) {...}
```

- `deposit(...)` and `mint(...)` – settle the pending reward, mint shares at the current price, add the assets to `accountedAssets` and forward them to the strategy. The caller is recorded in an append-only user set.
- `withdraw(...)` and `redeem(...)` – settle the pending reward, request the assets back from the strategy, burn the shares, subtract the assets from `accountedAssets` and transfer them to the receiver.

The settlement step, `_accrueRewards()`, computes the pending reward and compares the market balance plus `hiveStrategy.principal()` with `accountedAssets` plus the reward. If the backing is sufficient, it sends the reward to the strategy and adds it to `accountedAssets`. If not, the call reverts. Each settlement also updates the informational counters `totalRewardsAccrued` and `accRewardPerShare`, and `previewAccruedReward()` exposes the pending reward.

Anyone can add reward backing. Two view functions report the funding state:

```
function fundRewards(uint256 amount) external {...}
function rewardFundingStatus() external view returns (uint256 shortfall, bool sufficient) {...}
function availableExcessRewards() public view returns (uint256) {...}
```

`fundRewards(...)` transfers tokens into the market without a change to the share price. The tokens become backing for future settlements. `rewardFundingStatus()` returns the shortfall against the current requirement, and `availableExcessRewards()` returns the backing above it.

The owner operates the market through these functions:

```
function setApyBps(uint256 _apyBps, bool needAccrue) external onlyOwner nonReentrant {...}
function setStrategy(IHiveStrategy _hiveStrategy) external onlyOwner nonReentrant {...}
function accrue() external onlyOwner nonReentrant returns (uint256) {...}
function withdrawExcessRewards(address to, uint256 amount) external onlyOwner nonReentrant {...}
function rescueToken(IERC20 token, address to, uint256 amount) external onlyOwner nonReentrant {...}
function pause() external onlyOwner {...}
function unpause() external onlyOwner {...}
```

- `setApyBps(...)` – changes the annual rate. The `needAccrue` flag selects whether the pending reward is settled at the old rate first.
- `setStrategy(...)` – checks that the new strategy uses the same asset and points to this market, withdraws everything from the old strategy, and deposits `accountedAssets` into the new one. The zero address detaches the strategy.
- `accrue()` – settles the pending reward on demand.
- `withdrawExcessRewards(...)` – transfers backing that exceeds the current requirement, capped at `availableExcessRewards()`.
- `rescueToken(...)` – recovers any token other than the vault asset.
- `pause()` and `unpause()` – stop and resume all four user entry points.

The owner can also read the list of depositors through `getUserSetLength()`, `getAllUsers()` and `getUsers(...)`.

6.3 IHiveStrategy

`IHiveStrategy` is the interface between the market and the strategy that holds its assets:

```
interface IHiveStrategy {
    function principal() external view returns (uint256);
    function asset() external view returns (address);
    function market() external view returns (address);
    function pendingRewards() external view returns (uint256);
    function harvest() external;
    function retryDeposit() external;
    function deposit(uint256 assets) external;
    function withdraw(uint256 assets) external;
    function withdrawAll() external;
}
```

The market uses `deposit(...)`, `withdraw(...)` and `withdrawAll()` to move assets, `asset()` and `market()` to validate a new strategy, and `principal()` as the strategy part of its backing. `principal()` is a counter of the assets that the market deposited minus the assets it withdrew, so it does not include strategy yield. The market does not call `pendingRewards()`, `harvest()` or `retryDeposit()`. The operator manages the strategy yield off-chain.

6.4 Trust model

The owner is trusted. It sets the annual rate, chooses the strategy, pauses the market and withdraws excess reward backing. Users depend on the operator to keep the reward backing ahead of the accrued reward, because each settlement requires backing for the pending reward (see `_accrueRewards()`). The strategy is trusted to hold the principal, to report it through `principal()`, and to return the requested assets on withdrawal.

6.5 System Flow

a. `HiveMarket` is deployed with the asset, the initial rate and an owner address. The owner attaches a strategy with `setStrategy(...)`.

b. A user calls `deposit(...)` or `mint(...)`. The market settles the pending reward, mints shares and forwards the assets to the strategy.

c. Over time, the pending reward grows at `apyBps` and is visible through `totalAssets()`.

d. The operator harvests the strategy yield off-chain and sends it to the market with `fundRewards(...)`. The next settlement moves the recognized reward into the strategy and into `accountedAssets`.

e. A user calls `withdraw(...)` or `redeem(...)`. The market settles the pending reward, pulls the assets from the strategy, burns the shares and pays the receiver at the new share price.

7 Findings

7.1 Medium

7.1.1 [Medium] Underfunded reward accrual locks the principal of all users

File(s): HiveMarket.sol

Description: The HiveMarket contract pays a fixed annual rate (apyBps) on the assets it accounts for. The reward accrues linearly with time, and the operator backs it externally through fundRewards(...). Every user entry point (deposit(...), mint(...), withdraw(...) and redeem(...)) first calls _accrueRewards(), which recognizes the pending reward only if the market can back it:

```
function _accrueRewards() internal returns (uint256) {
    uint256 reward = previewAccruedReward();
    // ...
    uint256 _balance = IERC20(asset()).balanceOf(address(this));

    if (address(hiveStrategy) != address(0)) {
        _balance += hiveStrategy.principal();
    }

    // @audit reverts the whole call, including withdrawals of principal
    if (_balance < accountedAssets + reward) {
        revert InsufficientReward();
    }
    // ...
}
```

However, the function reverts when the backing is short, instead of recognizing only the part of the reward that is funded. All deposited principal sits in the strategy, so the backing is accountedAssets plus the reward funding held by the market. The pending reward grows every second while apyBps > 0 and totalSupply() > 0. As soon as the reward funding runs out, every deposit, mint, withdrawal and redemption reverts, also for principal that is fully backed. The same lock occurs in a funded market when the funding is used up, or right after the owner calls withdrawExcessRewards(...), which leaves the backing exactly at the current requirement.

Impact: Users cannot withdraw their principal until the operator adds more funding. The only way to unlock the market without new funding is setApyBps(0, false), which removes the pending reward that totalAssets() already showed to the users.

Recommendation(s): Do not revert in the settlement path. Cap the reward recognized at each settlement at backing - accountedAssets, and apply the same cap in previewAccruedReward(), so that previews and execution agree.

Status: Fixed

Update from the client: Fixed in 28838a3.

7.1.2 [Medium] The amount returned by `strategy.withdraw(...)` is not verified

File(s): `HiveMarket.sol`

Description: When a user calls `withdraw(...)` or `redeem(...)`, the market first pulls the assets back from the strategy through `_ensureLiquidity(...)` and then transfers them to the receiver:

```
function _ensureLiquidity(uint256 assets) internal {
    if (address(hiveStrategy) != address(0)) {
        // @audit the amount actually received is never checked
        hiveStrategy.withdraw(assets);
    }
}
```

However, the market does not check how many tokens arrive. A strategy can return less than requested without reverting, for example because of rounding in the underlying protocol, withdrawal fees, or low liquidity in a lending pool. The market then pays the difference from its own balance, which holds only the reward funding, while `accountedAssets` decreases by the full amount.

Impact: The reward funding is used to pay principal, so the lock described in the "Underfunded reward accrual" issue occurs sooner. Once the market balance is empty, withdrawals revert with a generic ERC20 error that does not show the cause. Small rounding shortfalls accumulate over many withdrawals.

Recommendation(s): Measure the amount that each strategy call delivers (`balanceAfter - balanceBefore`), and use one of these options. Option A: revert if the amount delivered is lower than requested, with a small tolerance. Many underlying protocols round down by 1 wei or more on each withdrawal, so a strict check would make every withdrawal revert. This is the correct behavior when the shortfall comes from low liquidity:

```
uint256 internal constant WITHDRAW_TOLERANCE = 10; // wei

function _ensureLiquidity(uint256 assets) internal returns (uint256 received) {
    if (address(hiveStrategy) == address(0)) return assets;
    uint256 balanceBefore = IERC20(asset()).balanceOf(address(this));
    hiveStrategy.withdraw(assets);
    received = IERC20(asset()).balanceOf(address(this)) - balanceBefore;
    if (received + WITHDRAW_TOLERANCE < assets) revert InsufficientLiquidity();
}
```

Option B: in `redeem(...)`, let the redeemer pay the shortfall. Burn the shares, pay only the amount that arrived and reduce `accountedAssets` by the full `assets`:

```
function redeem(uint256 shares, address receiver, address owner)
    public
    override
    nonReentrant
    whenNotPaused
    returns (uint256 assets)
{
    // ...
    _accrueRewards();
    assets = previewRedeem(shares);
    uint256 received = _ensureLiquidity(assets);
    // @audit the redeemer pays the shortfall, not the reward funding
    uint256 paid = Math.min(received, assets);
    _withdraw(msg.sender, receiver, owner, paid, shares);
    accountedAssets -= assets;
    return paid;
}
```

Option B handles fees and rounding without a revert. However, `withdraw(...)` must still use Option A, since EIP-4626 requires it to deliver exactly `assets`. Option B also requires the strategy to revert on low liquidity and to return less only for fees or rounding. Otherwise, the redeemer loses the part that stays in the strategy.

Status: Fixed

Update from the client: Fixed in `1375f44` and `21dd474`.

7.2 Low

7.2.1 [Low] `setApyBps(...)` with `needAccrue == false` applies the new rate retroactively

File(s): `HiveMarket.sol`

Description: The owner can change the annual rate with `setApyBps(...)`. The `needAccrue` flag controls whether the pending reward is settled at the old rate first:

```
function setApyBps(uint256 _apyBps, bool needAccrue) external onlyOwner nonReentrant {
    // @audit without settlement, lastAccrualTime is not updated
    if (needAccrue) {
        _accrueRewards();
    }

    uint256 _oldBps = apyBps;
    apyBps = _apyBps;
    emit ApyBpsChanged(_oldBps, _apyBps);
}
```

However, when `needAccrue` is `false`, `lastAccrualTime` stays unchanged. The next settlement computes the reward for the whole period since the last settlement with the new rate, including the time before the change. When the market is underfunded, `needAccrue == true` reverts, so the owner can only use `false`. The flag is also not reliable, because any user can settle at the old rate right before the owner transaction with a 1 wei deposit.

Impact: A rate decrease removes reward that `totalAssets()` and `previewRedeem(...)` already showed to users. A rate increase creates a retroactive liability that the reward funding did not plan for, which makes the market lock sooner.

Recommendation(s): Always settle the pending reward before changing the rate. If underfunding must not block the change, combine this with a settlement that caps the reward at the available backing.

Status: Fixed

Update from the client: Fixed in `e363748`.

7.2.2 [Low] `maxDeposit(...)`, `maxMint(...)`, `maxWithdraw(...)` and `maxRedeem(...)` ignore the pause and the solvency check

File(s): `HiveMarket.sol`

Description: `HiveMarket` inherits the OpenZeppelin `ERC4626` implementation and adds `whenNotPaused`, the reward solvency check and the strategy withdrawal to the deposit and withdrawal flows. EIP-4626 requires `maxDeposit(...)` and `maxMint(...)` to return 0 when deposits are disabled, and `maxWithdraw(...)` and `maxRedeem(...)` to return an amount that would not revert. However, the four `max*` functions are not overridden. They return `type(uint256).max` and the full balance of the owner even when the contract is paused, when `_accrueRewards()` would revert with `InsufficientReward`, or when the strategy cannot return the assets.

Impact: The market is not compliant with EIP-4626. Integrators and routers that rely on these functions build transactions that revert.

Recommendation(s): Override the four `max*` functions to return 0 when the contract is paused or underfunded, and to cap withdrawals at the liquidity available in the strategy.

Status: Fixed

Update from the client: Fixed in `cabffaa`.

7.3 Informational

7.3.1 [Info] Frequent settlement compounds rewards above the configured APY

File(s): `HiveMarket.sol`

Description: The `previewAccruedReward()` function computes the reward as simple interest on `accountedAssets` for the time since the last settlement:

```
function previewAccruedReward() public view returns (uint256) {
    // ...
    // @audit simple interest on accountedAssets, which includes earlier rewards
    return Math.mulDiv(accountedAssets, apyBps * elapsed, BPS * YEAR);
}
```

However, each settlement adds the reward to `accountedAssets`, so the next period also earns on it. The effective yield is `(1 + apy / n)^n - 1` for `n` settlements per year. Any user can trigger a settlement with a 1 wei deposit, and normal activity has the same effect. With a 10% APY and one settlement per day, a 1,000e18 deposit grows to about 1,105.16e18 after one year instead of 1,100e18.

Impact: The reward liability is up to about 5.2% higher than a funding plan based on simple interest at the configured rate, which brings the funding shortfall closer.

Recommendation(s): Document `apyBps` as a nominal rate that compounds at each settlement, or plan the reward funding for the continuous compounding upper bound.

Status: Acknowledged

Update from the client: Acknowledged.

7.3.2 [Info] One-step ownership transfer

File(s): `HiveMarket.sol`

Description: `HiveMarket` inherits OpenZeppelin `Ownable`. The owner controls the functions that the market needs to operate: `setApyBps(...)`, `setStrategy(...)`, `pause()`, `unpause()`, `accrue()` and `withdrawExcessRewards(...)`. However, `transferOwnership(...)` sets the new owner in a single call, without acceptance by the new owner. A wrong non-zero address moves ownership to an account that nobody controls. `renounceOwnership()` is also available and sets the owner to `address(0)`.

Impact: If ownership is lost, the APY and the strategy can no longer be changed. If the market is paused at that moment, deposits and withdrawals stay blocked permanently.

Recommendation(s): Inherit `Ownable2Step` instead of `Ownable`, so that the new owner must call `acceptOwnership()`. Also consider overriding `renounceOwnership()` to revert.

Status: Fixed

Update from the client: Fixed in `d9ba8e9`.

7.3.3 [Info] No upper bound on apyBps

File(s): HiveMarket.sol

Description: The annual rate is set in the constructor and changed with setApyBps(...). Neither validates the value:

```
function setApyBps(uint256 _apyBps, bool needAccrue) external onlyOwner nonReentrant {
    // ...
    // @audit no upper bound
    apyBps = _apyBps;
    // ...
}
```

A plausible input mistake, for example 100000 (1000%) instead of 1000 (10%), is accepted. The reward funding then runs out quickly and the market locks. Users can also withdraw at the inflated share price before the owner corrects the value, which uses up the reward funding of the other users.

Impact: An input mistake by the owner can lock the market or drain the reward funding before it is corrected.

Recommendation(s): Add a MAX_APY_BPS constant that matches the highest rate the protocol intends to pay, and revert in the constructor and in setApyBps(...) if the value is higher.

Status: Fixed

Update from the client: Fixed in 091028e.

7.3.4 [Info] Code hygiene suggestions

File(s): HiveMarket.sol, IHiveStrategy.sol

Description: The codebase contains several minor code quality points. None of them is a security issue on its own, but together they make the code harder to read, review and integrate with:

- The RewardPoolChanged event and the InvalidRewardAmount error are declared but never used;
- The event name ChnageStrategy contains a typo;
- The hiveStrategy state variable has no explicit visibility, so it is internal and has no getter. Users and integrators cannot read the current strategy address through the ABI;
- The getUserSetLength(), getAllUsers() and getUsers(...) functions are restricted to the owner, but the underlying storage is public on chain, so the restriction provides no privacy. getAllUsers() returns an unbounded array;
- _accrueRewards() checks that hiveStrategy is set before it calls _deployToStrategy(...), which performs the same check again;

Impact: No direct security impact. The points above reduce readability and maintainability, and the missing getter for hiveStrategy makes the market harder to monitor.

Recommendation(s): Consider applying the following changes:

- Remove RewardPoolChanged and InvalidRewardAmount;
- Rename ChnageStrategy to ChangeStrategy;
- Declare hiveStrategy as public;
- Remove the onlyOwner modifier from the userSet getters, or remove the getters and track users off-chain from events. Keep only the paginated getUsers(...);
- Remove the redundant hiveStrategy check in _accrueRewards();

Status: Fixed

Update from the client: Fixed in de3e01a.

8 Threat Model

Methodology

The threat model uses actor-capability analysis as its spine and cross-checks it against an invariant / violation matrix. The actor lens fits **HyperHive** because its risk does not come from complex permissionless logic. It comes from the parties the market must rely on: the owner, the operator who funds the rewards, and a strategy whose reports the market cannot verify. The matrix then lists each property the market should preserve, the way an attacker or a failure could break it, and what stands against that.

The model is deliberately un-tiered. It carries no severity or likelihood ratings; those belong to the findings in Section 7. Its purpose is to describe what can go wrong, who can cause it, and which assumption keeps it out of reach. The model describes the audited commit. Section 7 gives the status of each fix.

Out of Scope

The strategy implementation behind `IHiveStrategy` is out of scope. The review treats it as trusted and models only the way `HiveMarket` calls it and what the market believes about its answers. The OpenZeppelin v5 contracts (`ERC4626`, `Ownable`, `Pausable`, `ReentrancyGuard`, `SafeERC20`) are trusted primitives.

The yield harvest and the reward funding happen off-chain and are operational assumptions. The on-chain effect of the funding that the operator actually sends, or fails to send, is still modelled. The vault asset is assumed to be a standard ERC-20 token without transfer fees, rebasing or callbacks; the choice of asset is a deployment decision.

Trust Assumptions

The model rests on the following assumptions. The team confirmed each of them during the audit; the code does not enforce them.

- **No loss of principal in the strategy.** The team states that *there will be no loss of principal*. `principal()` changes only by the amounts that the market deposits and withdraws, and the strategy is trusted on that value. The market has no mechanism to detect a loss or to share it between holders. If the assumption fails, for example through an exploit of the underlying protocol or an operator mistake, the share price stays at the pre-loss value, the users who withdraw first receive full value, and the last users to withdraw absorb the whole loss.
- **Synchronous strategy.** The strategy runs on HyperEVM and returns the withdrawn assets in the same transaction.
- **Reward funding.** The operator harvests the strategy yield off-chain and keeps the reward funding ahead of the accrued reward.
- **Trusted owner.** The owner sets the rate, chooses the strategy and can pause the market.

Assets

The following are what an attacker would seek to divert, corrupt, or deny, ranked roughly by impact:

a. **User principal.** The assets that the strategy holds for the market, counted in `accountedAssets` and in `principal()`. The goal is to take them, or to make the market pay them out to the wrong holders.

b. **Withdrawal liveness.** The ability of every holder to call `withdraw(...)` or `redeem(...)` at any time the market is not paused. The goal is to block exits, so that funds stay locked in the market.

c. **Reward funding.** The asset balance that the market holds above `accountedAssets`, sent through `fundRewards(...)`. The goal is to use it for something other than rewards, such as covering a strategy shortfall or an early exit.

d. **Share price integrity.** The ratio of `totalAssets()` to `totalSupply()`. The goal is to move it, so that a deposit gets too many shares or a redemption gets too many assets.

e. **Reward rate integrity.** The promise that holders earn `apyBps` per year. The goal is to make the market pay more or less than that for a given period.

Trust Boundaries

- **`HiveMarket` ↔ strategy** (dominant boundary). The market sends all principal to the strategy and gives it an unlimited approval. It trusts `principal()` as the strategy's value and trusts `withdraw(...)` and `withdrawAll()` to return the requested amount. None of this is checked on-chain: `principal()` is a counter that does not show losses, and the market does not measure what a withdrawal delivers. The only on-chain control is the check in `setStrategy(...)` that the strategy uses the same asset and points to this market.

- **`HiveMarket` ↔ operator.** Each settlement requires the market balance plus `principal()` to cover `accountedAssets` plus the pending reward. The operator must keep the funding ahead of the accrued reward. Nothing on-chain enforces this, and when it fails, every user entry point reverts.
- **Users ↔ `HiveMarket`.** Users reach only the four ERC-4626 entry points, `fundRewards(...)` and the views. `nonReentrant` and `whenNotPaused` guard the entry points, and the share price follows `accountedAssets`, not the token balance, so a direct transfer cannot move it.
- **Owner ↔ `HiveMarket`.** `onlyOwner` gates every administrative function. `rescueToken(...)` refuses the vault asset, and `withdrawExcessRewards(...)` caps the amount at `availableExcessRewards()`. These bounds hold only as far as `principal()` is correct, and `setStrategy(...)` places no limit on what the new strategy may do.

Threat Actors

Each actor is described by its honest capabilities, its blast radius if compromised or mistaken, and the bounds that constrain it.

Users (permissionless)

Honest capabilities: deposit, mint, withdraw and redeem for themselves or, with a share allowance, for another owner. Anyone can also add reward funding through `fundRewards(...)`.

Blast radius: a user cannot move the share price, because donations do not change `accountedAssets`. A user who exits first after an unreported strategy loss receives the full share value, and the remaining holders absorb the loss. A user who exits when the strategy delivers less than requested takes the shortfall from the reward funding.

Bounds: `nonReentrant`, `whenNotPaused`, the ERC-4626 allowance checks, and the share price based on `accountedAssets`.

(Tampering / Denial of service.)

Operator (reward funder)

Honest capabilities: harvest the strategy yield off-chain and send it to the market with `fundRewards(...)`, so that each settlement is backed.

Blast radius: if the operator stops or falls behind, the next settlement reverts with `InsufficientReward`, and no user can deposit or withdraw principal until the market is funded again. The operator does not have to act against users for this to happen; a delay is enough.

Bounds: none on-chain. `rewardFundingStatus()` shows the shortfall, but the market does not cap a settlement at the available backing.

(Denial of service.)

Strategy

Honest capabilities: hold the principal, return it synchronously on `withdraw(...)` and `withdrawAll()`, and report deposits minus withdrawals through `principal()`.

Blast radius: the strategy holds all principal and an unlimited approval over the market balance. A loss inside the strategy stays invisible to the market, and a short delivery is paid from the reward funding or makes the withdrawal fail. A malicious or compromised strategy can take every asset that the market holds or has deposited.

Bounds: only the asset and market checks in `setStrategy(...)`. The market has no way to verify the strategy's value or the amount it delivers.

(Tampering / Information disclosure / Elevation of privilege.)

Owner (fully trusted)

Honest capabilities: set the reward rate, attach and replace the strategy, settle on demand, pause and unpause, withdraw excess funding, and recover non-asset tokens.

Blast radius: the owner is a trust assumption, not a modelled attacker. A mistaken `setApyBps(...)` with `needAccrue == false` applies the new rate to the whole period since the last settlement, and `apyBps` has no upper bound. A compromised owner key can attach a malicious strategy and take all principal, or pause the market indefinitely. Ownership moves in one step, so a wrong address in `transferOwnership(...)` cannot be undone.

Bounds: `rescueToken(...)` refuses the vault asset, and `withdrawExcessRewards(...)` cannot go below the counted requirement. There is no timelock and no limit on the rate or the strategy choice.

(Elevation of privilege / Tampering / Denial of service.)

Invariant / Violation Matrix

Table 4: Invariants of `HiveMarket`, how they could be violated, and what stands against each violation.

Invariant	How it could be violated	Mitigation / bound
The real backing covers `accountedAssets`.	A strategy loss, or a short delivery on withdrawal.	Partial. The settlement check uses `principal()`, a counter that does not show losses; the delivered amount is not checked.
A holder can exit with their principal while the market is not paused.	The reward funding falls behind the accrued reward, and every settlement reverts.	None on-chain. It depends on the operator funding the market on time.
Losses are shared pro rata by all holders.	The first holders to exit after an unreported loss receive full value.	None on-chain. It depends on the trust assumption that the strategy never loses principal.
Reward funding pays only rewards.	A short strategy delivery or a migration shortfall is paid from the market balance.	None. The market pays the receiver from its balance without checking where the assets came from.
A direct transfer cannot move the share price.	Donating the asset to the market or the strategy before a victim's deposit.	Holds. `totalAssets()` uses `accountedAssets`, not the token balance.
Holders earn `apyBps` per year.	Frequent settlements compound the rate; a rate change without settlement applies retroactively.	Partial. `setApyBps(...)` can settle first, but only when `needAccrue` is set.
The owner cannot take user principal.	`rescueToken(...)` or `withdrawExcessRewards(...)` on the vault asset; a new malicious strategy.	Partial. The two functions are bounded; `setStrategy(...)` relies on the owner being trusted.
A paused market accepts no deposits or withdrawals.	A user or integrator calls an entry point during the pause.	Holds for the four entry points; the `max*` views still report non-zero limits.

9 Evaluation of Provided Documentation

The **HyperHive** team provided one document, `HiveMarket.md`, which describes the functionality and the asset accounting of `HiveMarket`. The repository at the audited commit contains only the two Solidity files and a small number of code comments.

- **External documentation:** `HiveMarket.md` explains the share model, the reward formula and the accrual steps, the reward funding functions, each user operation, the administrative functions, the depositor address set and the events. It also states that `setApyBps(...)` with `needAccrue == false` applies the new rate to the whole period since the last settlement. The descriptions match the code in most places.
- **In-code documentation:** minimal. `HiveMarket.sol` has ten comment lines in total, of which four are NatSpec tags. No function documents its parameters, return values, access control or revert conditions. `IHiveStrategy.sol` has no comments at all.

The document covers only `HiveMarket`. It does not describe `IHiveStrategy`, the pause, the strategy migration in `setStrategy(...)` or the trust assumptions. The following properties had to be confirmed with the team during the audit:

- the strategy runs on HyperEVM and returns assets synchronously, in the same transaction;
- `principal()` is a counter that changes only by the amounts that the market deposits and withdraws;
- the operator harvests the strategy yield off-chain and funds the rewards through `fundRewards(...)`;
- the owner is a trusted party.

Overall, the documentation is rated **Medium**. The document gives a clear functional description of the market, but it presents the backing check as a safeguard without its consequence: after the fix, a reward that is not backed at settlement is dropped, so users depend on the operator to fund the rewards on time.

9.1 Documentation QA

The following discrepancies between the documentation (`HiveMarket.md` and the code comments) and the behaviour of `HiveMarket` were identified during the documentation QA pass.

- The contract-level `@notice` describes a *configured annual reward rate*. `HiveMarket.md` notes that subsequent rewards use the updated `accountedAssets`, but neither states that the effective yield is therefore above `apyBps` when settlements are frequent.
- The fix-review commit changes only `HiveMarket.sol`, so `HiveMarket.md` now describes the code at the audited commit. It still documents `setApyBps(newApyBps, needAccrue)` and the `InsufficientReward` revert on settlement. It does not describe the settlement cap at the available backing, the strategy withdrawal tolerance and `setStrategy(...)` with its tolerance parameter, the `maxApyBps` bound or the two-step ownership transfer.

Recommendations

CODESPECT recommends:

a. Extend `HiveMarket.md` with the reward funding model, the role of the operator, the trust assumptions, the pause and the strategy migration.

b. Document `IHiveStrategy`, in particular the exact meaning of `principal()` and the guarantees of `withdraw(...)` and `withdrawAll()` (synchronous delivery, behaviour on low liquidity).

c. Add NatSpec to every external function of `HiveMarket`, including access control and revert conditions such as `InsufficientLiquidity` and `ApyBpsExceedsMaximum`.

d. Update `HiveMarket.md` for the changes made during the fix review, and correct the statements listed in the Documentation QA above.

10 Test Suite Evaluation

The repository at the audited commit contains no test suite and no build configuration (no `foundry.toml`, Hardhat configuration or dependency manifest). To compile the code, **CODESPECT** used Foundry with Solidity 0.8.28 and OpenZeppelin Contracts v5.1.0, the versions compatible with the pragmas and imports in scope.

10.1 Compilation output

```
> forge build
Compiling 23 files with Solc 0.8.28
Solc 0.8.28 finished in 96.36ms
Compiler run successful!
```

The code compiles without errors or warnings with this setup.

10.2 Tests Output

```
> forge test
No tests found in project! Forge looks for functions that start with `test`
```

The project provides no tests, so no part of the in-scope code is exercised.

10.3 Cross-Cutting Weaknesses

- **No tests.** No unit, integration, fuzz or invariant test exists for `HiveMarket` or for any strategy.
- **No build setup.** The repository does not pin the compiler version or the OpenZeppelin version, so the build cannot be reproduced from the repository alone.
- **No strategy mock or reference implementation.** The interaction between the market and `IHiveStrategy` cannot be tested without an implementation of the interface.

10.4 Specific Untested Paths

All paths of the in-scope code are untested. The most important ones are:

- **Reward accrual:** `_accrueRewards()` with sufficient, exactly sufficient and insufficient backing, and the resulting `InsufficientReward` revert on every entry point.
- **Share accounting:** `deposit(...)`, `mint(...)`, `withdraw(...)` and `redeem(...)` at a share price that includes pending rewards, and the rounding between them.
- **Strategy interaction:** the amount that the strategy delivers on `IHiveStrategy.withdraw(...)` and the migration in `setStrategy(...)`.
- **Owner operations:** `setApyBps(...)` with and without settlement, `withdrawExcessRewards(...)`, `rescueToken(...)` and the pause.
- **Long-running behaviour:** the effective yield and the reward liability over many settlements.

10.5 Conclusion

The main gaps are:

a. no tests of any kind;

b. no reproducible build setup;

c. no strategy mock or reference implementation.

The test suite is rated **Low**. Several issues in this report, for example the funding lock, would have been visible in basic tests with a mock strategy.

To close them, **CODESPECT** recommends:

- Add a Foundry project with a pinned compiler and pinned dependencies.
- Add unit tests for every external function, including the revert paths.

- Add invariant tests for the solvency of the market: the backing must cover `accountedAssets` plus the pending reward.
- Add fork tests against the production strategy before deployment.